

# Tarea 1 Cadenas de Markov - Simulación de Monte Carlo

## Integrantes

- Nicolás Betancur Sánchez 1010023513
- Andrés Felipe Gómez Durán 1001298960
- Germán Camilo Rodríguez Perilla 1032480170

## Lenguaje o Software implementado

Todas las simulaciones se realizaron en el lenguaje de programación **python**.

## Planteamiento general

Se considera el espacio muestral

$$\Omega = [-2, 2] \times [-2, 2],$$

cuya área es  $|\Omega| = 16$ . Si un punto  $(X, Y)$  se distribuye uniformemente en  $\Omega$ , la probabilidad de pertenecer a un conjunto medible  $A \subseteq \Omega$  es

$$P((X, Y) \in A) = \frac{|A|}{|\Omega|}.$$

## Método de simulación

Se emplea el método de Monte Carlo: se generan puntos pseudoaleatorios uniformes en  $\Omega$ , se registra cuántos satisfacen la condición de pertenencia al círculo y se usa la frecuencia relativa como estimación de la probabilidad. Para que los resultados puedan reproducirse, se fija la semilla 20260909.

### 1. Aproximar la probabilidad $P(C_1)$

Sea

$$C_1 = \{(x, y) \in \Omega : x^2 + y^2 \leq 1\}.$$

#### 1.1 Generación de puntos uniformes

Para cada tamaño de muestra  $n$ , se generan pares independientes

$$X_j \sim \text{Unif}(-2, 2), \quad Y_j \sim \text{Unif}(-2, 2), \quad j = 1, \dots, n.$$

La independencia de las coordenadas hace que  $(X_j, Y_j)$  sea uniforme en todo el cuadrado  $\Omega$ .

#### 1.2 Identificación de los puntos dentro del círculo

Para cada punto se calcula la variable indicadora

$$I_j = \mathbf{1} \{X_j^2 + Y_j^2 \leq 1\}.$$

Así,  $I_j = 1$  si el punto está dentro de  $C_1$  y  $I_j = 0$  en caso contrario. El número de puntos dentro del círculo es  $N_1 = \sum_{j=1}^n I_j$ . Cada  $I_j$  será una variable aleatoria Bernoulli con probabilidad de éxito  $P(C_1)$ .

### 1.3 Estimación de la probabilidad y del error

La frecuencia relativa

$$\hat{P}_n(C_1) = \frac{N_1}{n} = \frac{1}{n} \sum_{j=1}^n I_j$$

es un estimador insesgado de  $P(C_1)$ . Como el círculo unitario tiene área  $\pi$  y el cuadrado tiene área 16,

$$P(C_1) = \frac{\pi}{16} \approx 0.19634954.$$

El error absoluto se calcula mediante

$$\left| P(C_1) - \hat{P}_n(C_1) \right|.$$

Por la ley de los grandes números,  $\hat{P}_n(C_1) \rightarrow P(C_1)$  cuando  $n \rightarrow \infty$ .

### 1.4 Tamaños de muestra

La simulación se realiza para

$$n \in \{10^2, 10^3, 10^4, 10^5, 10^6\}.$$

Para cada valor se reportan  $n$ , el número de puntos dentro del círculo,  $\hat{P}_n(C_1)$  y el error absoluto.

### 1.5 Implementación y resultados

```
from random import Random
from math import pi, sqrt

semilla = 20260909
tamanos = [10**i for i in range(2, 7)]
p_c1 = pi / 16
resultados_c1 = []

print(f'{n':>10} {'Dentro de C1':>15} {'P estimada':>15} {'Error':>15}")

for indice, n in enumerate(tamanos):
    generador = Random(semilla + indice)
    puntos_dentro = 0

    for _ in range(n):
        # Genera los puntos aleatorios uniformes
        x = generador.uniform(-2, 2)
        y = generador.uniform(-2, 2)

        if x*x + y*y <= 1:
            # De una vez acumula la cantidad de puntos en el círculo
            puntos_dentro += 1

    p_estimada = puntos_dentro / n
    # Frecuencia

    error = abs(p_c1 - p_estimada)
    resultados_c1.append((n, puntos_dentro, p_estimada, error))
```

```
print(
    f"{n:>10} {puntos_dentro:>15} "
    f"{p_estimada:>15.8f} {error:>15.8f}"
)
```

| n       | Dentro de C1 | P estimada | Error      |
|---------|--------------|------------|------------|
| 100     | 20           | 0.20000000 | 0.00365046 |
| 1000    | 194          | 0.19400000 | 0.00234954 |
| 10000   | 1947         | 0.19470000 | 0.00164954 |
| 100000  | 19432        | 0.19432000 | 0.00202954 |
| 1000000 | 197054       | 0.19705400 | 0.00070446 |

## 1.6 Tabla de resultados

| $n$    | Puntos dentro de $C_1$ | $\hat{P}_n(C_1)$ | Error absoluto |
|--------|------------------------|------------------|----------------|
| $10^2$ | 20                     | 0.20000000       | 0.00365046     |
| $10^3$ | 194                    | 0.19400000       | 0.00234954     |
| $10^4$ | 1 947                  | 0.19470000       | 0.00164954     |
| $10^5$ | 19 432                 | 0.19432000       | 0.00202954     |
| $10^6$ | 197 054                | 0.19705400       | 0.00070446     |

## 1.7 Análisis de los resultados

Las estimaciones se concentran alrededor del valor teórico  $\pi/16 \approx 0.19634954$  a medida que aumenta  $n$ . La reducción del error no tiene que ser monótona: cada estimación contiene variación aleatoria y, por casualidad, una muestra menor puede quedar más cerca del valor real que una muestra mayor. Lo relevante es que la dispersión típica disminuye a la tasa  $1/\sqrt{n}$ , puesto que

$$\text{Var}(\hat{P}_n(C_1)) = \frac{P(C_1)[1 - P(C_1)]}{n}.$$

Con  $n = 10^6$ , el error observado es aproximadamente  $7.04 \times 10^{-4}$ , lo cual evidencia que la frecuencia relativa se mantiene cerca del valor teórico.

## 2. Aproximar el número $\pi$

### 2.1 Relación entre $P(C_1)$ y $\pi$

La relación geométrica obtenida en el ejercicio anterior es

$$P(C_1) = \frac{\text{área del círculo unitario}}{\text{área del cuadrado}} = \frac{\pi}{16}.$$

Al despejar  $\pi$  se obtiene

$$\pi = 16P(C_1).$$

## 2.2 Construcción y justificación del estimador

Al reemplazar la probabilidad desconocida por su frecuencia relativa se define

$$\hat{\pi}_n = 16\hat{P}_n(C_1).$$

Este estimador es insesgado, porque

$$E[\hat{\pi}_n] = 16E[\hat{P}_n(C_1)] = 16P(C_1) = \pi.$$

También es consistente: como  $\hat{P}_n(C_1) \rightarrow P(C_1)$  por la ley de los grandes números, entonces  $\hat{\pi}_n \rightarrow \pi$ .

## 2.3 Cálculo del error

Para cada tamaño de muestra se calcula

$$|\pi - \hat{\pi}_n| = \left| \pi - 16\hat{P}_n(C_1) \right|.$$

Se reutilizan exactamente las probabilidades estimadas en el ejercicio 1, tal como solicita el enunciado.

## 2.4 Implementación y resultados

```
resultados_pi = []

print(f"{'n':>10} {'P estimada':>15} {'Pi estimado':>15} {'Error':>15}")

for n, puntos_dentro, p_estimada, error_p in resultados_c1:
    pi_estimado = 16 * p_estimada      # Aproximaciones de pi
    error_pi = abs(pi - pi_estimado)
    resultados_pi.append((n, p_estimada, pi_estimado, error_pi))

print(
    f"{'n':>10} {'p_estimada':>15.8f} "
    f"{'pi_estimado':>15.8f} {'error_pi':>15.8f}"
)
```

| n       | P estimada | Pi estimado | Error      |
|---------|------------|-------------|------------|
| 100     | 0.20000000 | 3.20000000  | 0.05840735 |
| 1000    | 0.19400000 | 3.10400000  | 0.03759265 |
| 10000   | 0.19470000 | 3.11520000  | 0.02639265 |
| 100000  | 0.19432000 | 3.10912000  | 0.03247265 |
| 1000000 | 0.19705400 | 3.15286400  | 0.01127135 |

## 2.5 Tabla de resultados

| $n$    | $\hat{P}_n(C_1)$ | $\hat{\pi}_n$ | Error absoluto |
|--------|------------------|---------------|----------------|
| $10^2$ | 0.20000000       | 3.20000000    | 0.05840735     |
| $10^3$ | 0.19400000       | 3.10400000    | 0.03759265     |
| $10^4$ | 0.19470000       | 3.11520000    | 0.02639265     |
| $10^5$ | 0.19432000       | 3.10912000    | 0.03247265     |
| $10^6$ | 0.19705400       | 3.15286400    | 0.01127135     |

## 2.6 Análisis de los resultados

El factor 16 transforma directamente el error de la probabilidad:

$$|\pi - \hat{\pi}_n| = 16 \left| P(C_1) - \hat{P}_n(C_1) \right|.$$

Por eso las aproximaciones de  $\pi$  siguen el mismo patrón de convergencia observado en el ejercicio 1. El error puede aumentar ocasionalmente al pasar de un tamaño de muestra a otro, como ocurre entre  $10^4$  y  $10^5$ , sin contradecir la convergencia del método. En esta simulación, la aproximación más precisa se obtiene con  $n = 10^6$ :

$$\hat{\pi}_{10^6} = 3.152864, \quad |\pi - \hat{\pi}_{10^6}| \approx 0.01127135.$$

## 3. Considerar los conjuntos $C_{\sqrt{2}}$ y $C_2$

Se definen

$$C_{\sqrt{2}} = \{(x, y) \in \Omega : x^2 + y^2 \leq 2\}, \quad C_2 = \{(x, y) \in \Omega : x^2 + y^2 \leq 2^2\}.$$

### 3.1 Probabilidades teóricas

Para un círculo de radio  $r \leq 2$  contenido en  $\Omega$ , la probabilidad de pertenencia es

$$P(C_r) = \frac{\pi r^2}{16}.$$

En particular,

$$P(C_{\sqrt{2}}) = \frac{\pi(\sqrt{2})^2}{16} = \frac{\pi}{8} \approx 0.39269908,$$

y

$$P(C_2) = \frac{\pi(2)^2}{16} = \frac{\pi}{4} \approx 0.78539816.$$

### 3.2 Identificación de los puntos

Para cada radio se utiliza la variable indicadora

$$I_j^{(r)} = \mathbf{1} \{X_j^2 + Y_j^2 \leq r^2\}.$$

La estimación de la probabilidad es

$$\hat{P}_n(C_r) = \frac{1}{n} \sum_{j=1}^n I_j^{(r)}.$$

### 3.3 Estimador general de $\pi$

De  $P(C_r) = \pi r^2/16$  se despeja  $\pi$  y se obtiene

$$\hat{\pi}_n(r) = \frac{16}{r^2} \hat{P}_n(C_r).$$

El estimador es insesgado y consistente. Además,

$$\text{Var}(\hat{\pi}_n(r)) = \frac{256}{r^4} \frac{P(C_r)[1 - P(C_r)]}{n} = \frac{16\pi/r^2 - \pi^2}{n}.$$

Esta varianza disminuye al aumentar  $r$  dentro del intervalo considerado, lo cual anticipa que el radio  $r = 2$  debe proporcionar, en promedio, estimaciones más precisas.

### 3.4 Comparación de los tres radios

Para cada  $n$  se reinicia el generador con la misma semilla utilizada en el ejercicio 1. Así se vuelven a generar exactamente los mismos puntos y se comparan los radios 1,  $\sqrt{2}$  y 2 sobre una muestra común.

### 3.5 Implementación y resultados

```
resultados_ej3 = []

for indice, n in enumerate(tamanos):
    generador = Random(semilla + indice)
    dentro_r1 = 0
    dentro_raiz2 = 0
    dentro_r2 = 0

    for _ in range(n):
        x = generador.uniform(-2, 2)
        y = generador.uniform(-2, 2)
        distancia2 = x*x + y*y

        if distancia2 <= 1:
            dentro_r1 += 1
        if distancia2 <= 2:
            dentro_raiz2 += 1
        if distancia2 <= 4:
            dentro_r2 += 1

    p_r1 = dentro_r1 / n
    p_raiz2 = dentro_raiz2 / n
    p_r2 = dentro_r2 / n

    pi_r1 = 16 * p_r1
    pi_raiz2 = 8 * p_raiz2
    pi_r2 = 4 * p_r2

    resultados_ej3.append(
        (n, dentro_raiz2, p_raiz2, pi_raiz2,
         dentro_r2, p_r2, pi_r2, pi_r1)
    )

print("Resultados para los radios sqrt(2) y 2")
```

```

print(
    f"{'Radio':>8} {'n':>10} {'Dentro':>10} "
    f"{'P estimada':>12} {'Pi estimado':>12} {'Error':>12}"
)

for fila in resultados_ej3:
    n, d_raiz2, p_raiz2, pi_raiz2, d_r2, p_r2, pi_r2, pi_r1 = fila

    print(
        f"{'sqrt(2)':>8} {'n':>10} {'d_raiz2':>10} "
        f"{'p_raiz2':>12.8f} {'pi_raiz2':>12.8f} "
        f"{'abs(pi-pi_raiz2)':>12.8f}"
    )
    print(
        f"{'2':>8} {'n':>10} {'d_r2':>10} "
        f"{'p_r2':>12.8f} {'pi_r2':>12.8f} "
        f"{'abs(pi-pi_r2)':>12.8f}"
    )

print("\nComparación de las aproximaciones de Pi")
print(
    f"{'n':>10} {'r=1':>15} {'r=sqrt(2)':>15} {'r=2':>15}"
)

for fila in resultados_ej3:
    n, d_raiz2, p_raiz2, pi_raiz2, d_r2, p_r2, pi_r2, pi_r1 = fila
    print(
        f"{'n':>10} {'pi_r1':>15.8f} "
        f"{'pi_raiz2':>15.8f} {'pi_r2':>15.8f}"
    )

```

Resultados para los radios sqrt(2) y 2

| Radio   | n       | Dentro | P estimada | Pi estimado | Error      |
|---------|---------|--------|------------|-------------|------------|
| sqrt(2) | 100     | 42     | 0.42000000 | 3.36000000  | 0.21840735 |
| 2       | 100     | 80     | 0.80000000 | 3.20000000  | 0.05840735 |
| sqrt(2) | 1000    | 401    | 0.40100000 | 3.20800000  | 0.06640735 |
| 2       | 1000    | 799    | 0.79900000 | 3.19600000  | 0.05440735 |
| sqrt(2) | 10000   | 3890   | 0.38900000 | 3.11200000  | 0.02959265 |
| 2       | 10000   | 7806   | 0.78060000 | 3.12240000  | 0.01919265 |
| sqrt(2) | 100000  | 39130  | 0.39130000 | 3.13040000  | 0.01119265 |
| 2       | 100000  | 78476  | 0.78476000 | 3.13904000  | 0.00255265 |
| sqrt(2) | 1000000 | 392828 | 0.39282800 | 3.14262400  | 0.00103135 |
| 2       | 1000000 | 785840 | 0.78584000 | 3.14336000  | 0.00176735 |

Comparación de las aproximaciones de Pi

| n       | r=1        | r=sqrt(2)  | r=2        |
|---------|------------|------------|------------|
| 100     | 3.20000000 | 3.36000000 | 3.20000000 |
| 1000    | 3.10400000 | 3.20800000 | 3.19600000 |
| 10000   | 3.11520000 | 3.11200000 | 3.12240000 |
| 100000  | 3.10912000 | 3.13040000 | 3.13904000 |
| 1000000 | 3.15286400 | 3.14262400 | 3.14336000 |

### 3.6 Resultados para $r = \sqrt{2}$ y $r = 2$

| Radio      | $n$    | Puntos dentro | $\hat{P}_n(C_r)$ | $\hat{\pi}_n(r)$ | Error absoluto |
|------------|--------|---------------|------------------|------------------|----------------|
| $\sqrt{2}$ | $10^2$ | 42            | 0.42000000       | 3.36000000       | 0.21840735     |
| $\sqrt{2}$ | $10^3$ | 401           | 0.40100000       | 3.20800000       | 0.06640735     |
| $\sqrt{2}$ | $10^4$ | 3 890         | 0.38900000       | 3.11200000       | 0.02959265     |
| $\sqrt{2}$ | $10^5$ | 39 130        | 0.39130000       | 3.13040000       | 0.01119265     |
| $\sqrt{2}$ | $10^6$ | 392 828       | 0.39282800       | 3.14262400       | 0.00103135     |
| 2          | $10^2$ | 80            | 0.80000000       | 3.20000000       | 0.05840735     |
| 2          | $10^3$ | 799           | 0.79900000       | 3.19600000       | 0.05440735     |
| 2          | $10^4$ | 7 806         | 0.78060000       | 3.12240000       | 0.01919265     |
| 2          | $10^5$ | 78 476        | 0.78476000       | 3.13904000       | 0.00255265     |
| 2          | $10^6$ | 785 840       | 0.78584000       | 3.14336000       | 0.00176735     |

### 3.7 Tabla comparativa de las aproximaciones de $\pi$

| $n$    | $\hat{\pi}_n(r = 1)$ | $\hat{\pi}_n(r = \sqrt{2})$ | $\hat{\pi}_n(r = 2)$ |
|--------|----------------------|-----------------------------|----------------------|
| $10^2$ | 3.20000000           | 3.36000000                  | 3.20000000           |
| $10^3$ | 3.10400000           | 3.20800000                  | 3.19600000           |
| $10^4$ | 3.11520000           | 3.11200000                  | 3.12240000           |
| $10^5$ | 3.10912000           | 3.13040000                  | 3.13904000           |
| $10^6$ | 3.15286400           | 3.14262400                  | 3.14336000           |

### 3.8 Comparación y análisis

Los tres estimadores convergen a  $\pi$ , pero no tienen la misma variabilidad. La expresión

$$\text{Var}(\hat{\pi}_n(r)) = \frac{16\pi/r^2 - \pi^2}{n}$$

muestra que, para un mismo  $n$ , la varianza es menor cuando el radio es mayor. En particular, las desviaciones estándar teóricas son aproximadamente

$$\frac{6.36}{\sqrt{n}} \quad (r = 1), \quad \frac{3.91}{\sqrt{n}} \quad (r = \sqrt{2}), \quad \frac{1.64}{\sqrt{n}} \quad (r = 2).$$

Por tanto,  $r = 2$  es teóricamente la opción más eficiente entre los tres radios. En esta realización, el estimador con  $r = 2$  obtiene el menor error para  $n = 10^4$  y  $n = 10^5$ . Para  $n = 10^6$ , el estimador con  $r = \sqrt{2}$  resulta accidentalmente más cercano, con un error de 0.00103135, frente a 0.00176735 para  $r = 2$ .

Una realización aislada puede favorecer a otro radio por variación aleatoria. Sin embargo,  $r = 2$  utiliza una mayor proporción del cuadrado y aplica el menor factor de amplificación,  $16/r^2 = 4$ , por lo que ofrece mejores aproximaciones en promedio.